Date 18/12/25

UNIT-1

Testing

Testing is a method of estimating the product and quality of software. It is developed by providing the necessary details about the software efficient, portability, usability etc.

→ Testing is not only performed program execution but also defect bugs (errors) while execution of a program.

The Goals for Testing

The 2 major goals of testing are as follows.

1. Bug prevention
2. Bug discovered

Bug Prevention:- It is considered as the primary goal for testing when bug is detected appropriate method should be used to remove it.

- If bugs are not prevent then certain symptoms are discovered that are the major causes for the bugs.
- Bug prevention won't affect the execution schedule of the program if testing is performed at every stage of software development (design, coding)
- Bug prevention is a static one.

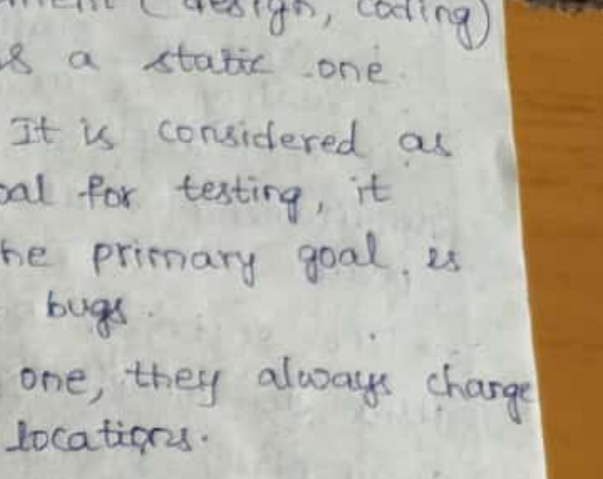
Bug discovered :- It is considered as the secondary goal for testing, it performs when the primary goal is failed to prevent bugs.

→ It is dynamic one, they always change their states or locations.

The Purpose of Testing

S/w Testing = S/w verification + S/w Validation

(a) For improving and assuring S/w Quality
(b) For verification and validation
(c) For estimating reliability



- Software testing is a process identification of errors and bugs.
- The aim of testing is more than just debugging and detecting bugs.

We have 3 types of purpose of testing

1. Software Quality assurance

→ Quality is used to determine if all the software meets all the requirements that are specified during phase.

→ The Software Quality Assurance is used to monitoring the software Engineering process and methods for ensuring its quality.

→ Requirements and Design and develop meets the software quality assurance

→ The software Quality assurance is very much related to quality assurance in the product manufacturing.

⑥ Verification and validation :-

→ Verification and validation is performed to test the quality of a software, based on the test results, testers makes decisions weather the product will work properly or not.

→ The specification and the test results of different products are compared to know which product has better quality

→ The quality is not tested directly but related aspects of quality has the following 3 aspects such as functionality, engineering and adoptability

→ These aspects are used to measure the scope of software quality

i) functionality :-

→ It is an exterior quality factor, it is determined by considering correctness, reliability and integrity of a product.

ii) Engineering :-

→ It is a interior quality factor, it is determined by considering efficiency, testability, documentation of structure of product.

iii) Adaptability :-

It is future quality factor it is determi-ned by consistency flexibility, reusability and maintainability of product.

→ When a validation test is performed on a product it is referred to as either a clean test or a positive test.

Estimating Reliability:-
Testing is important factor to quality of software reliability

→ S/w reliability is related to the different aspects of s/w such as a structure, documentation etc.

Productivity and Quality in S/w:-

→ In production of consumer goods and other products, every manufacturing stage is subjected to quality control and testing from component to final state.

→ If flaws are discovered at any stage the product is either discarded or cycled back for rework and correctness.

→ Testing and Test design are parts of quality assurance should also focus on bug prevention

→ A prevented bug is better than a defected and corrected bug.

Phases in a Tester's Mental life
5 phases.

1. Phase 0:- There is no difference between testing and debugging. 1956 (Debugging oriented)

2. Phase 1:- The purpose of testing here is to show that software works - 1978 (Demonstration oriented)

3. Phase 2:- (1979-1982) Destruction oriented
The purpose of testing is to show that software doesn't work.

4. Phase 3:- (1983-1987) Evaluation oriented
The purpose of testing is not to prove anything but to reduce the perceived risk of not working to an acceptable, applicable to large s/w produce.

5. phase 4:- (1988-2000) prevention oriented
one reason is to reduce the labour of testing

phases in a tester's mental life

phases in a tester's mental life can be categorized into the following 5 phases:

1. phase 0: (until 1956: Debugging oriented) There is no difference between testing and debugging. Phase 0 thinking was the norm in early days of software development till testing emerged as a discipline.

2. Phase 1: (1957-1978: Demonstration oriented) the purpose of testing here is to show that software works. Highlighted during the late 1970s. This failed because the probability of showing that software works 'decreases' as testing increases.

i.e., the more you test, the more likely you will find a bug.

3. Phase 2: (1979- 1982: Destruction oriented) the purpose of testing is to show that software doesn't work. This also failed because the software will never get released as you find one bug or the other. Also, a bug corrected may also lead to another bug.

4. Phase 3: (1983-1987: Evaluation oriented) The purpose of testing is not to prove anything but to reduce the perceived risk of not working to an acceptable value (statistical quality control) Notion is that testing does improve the product to the extent that testing catches bugs and to the extent that those bugs are fixed. The product is released when the confidence on that product is high enough. (Note: This is applied to large software products with millions of code and years of use.)

5. Phase 4: (1988-2000: prevention oriented) Testability is the factor considered here. One reason is to reduce the labour of testing. Other reason is to check the testable and non-testable code. Testable code has fewer bugs than the code that's hard to test. Identifying the testing techniques to test ~~the~~ code is the main key here.

We know that the software code must be designed and tested, but many appear to be unaware that tests themselves must be designed and tested. Tests should be properly designed and tested before applying it to the actual code.

Testing isn't everything:-

There are approaches other than testing to create better software. Methods other than testing include:

1. Inspection Methods:- Methods like walkthroughs, desk checking, formal inspections and code reading appear to

be as effective as testing but the bugs caught don't completely overlap.

2. Design style :- while designing the software itself, adopting stylistic objectives such as testability, openness and clarity can do much to prevent bugs.

3. Static Analysis Methods :- Includes formal analysis of source code during compilation. In earlier days, it is a routine job of the programmer to do that. Now, the compilers have taken over that job.

4. Languages :- The source language can help reduce certain kinds of bugs. Programmers find new bugs while using new languages.

5. Development Methodologies and Development Environment : The development process and the environment in which that methodology is embedded can prevent many kinds of bugs.

Date 20/12/25

White box Testing and Black box testing

White box testing / clear box, glass box, transparent box / structured testing

Definition

White box testing is a software testing technique in which the internal code structure, logic and paths of the program are tested.

Key points

- used in unit testing & integration testing
1. code is visible to the tester
- Focuses on how the program works
- Requires programming knowledge
- Mostly done by developers
- used mainly in Unit testing

Types

1. statement coverage — Every statement is executed atleast once.
2. Branch coverage — Every decision (true/false) is tested.
3. path coverage — All possible execution paths are tested.
4. condition coverage — Each condition is tested.
5. Loop testing :- Loops tested for 0, 1 and many iterations

Advantages

- Finds hidden code errors
- Improves code quality

Disadvantages

- Time consuming
- Needs coding skills

Black box Testing

Definition :-

Black box testing is a software testing technique where the internal code is not known, and testing is done using inputs and outputs.

Key points

- code is not visible
- Focuses on what the program does
- No programming knowledge required
- Done by testers and users
- used in system and Acceptance Testing.

Types

1. Functional Testing – Tests functions as per requirements
2. Non-Functional Testing :- Performance, security, usability
3. Boundary value Analysis (BVA) – Tests boundary values
4. Equivalence Partitioning – Divides inputs into valid/invalid sets.
5. Decision Table Testing – Tests combinations of inputs
6. State Transition Testing :- Tests different states of system.

Advantages

- user-focused testing
- simple and effective

Disadvantages

- Internal errors may be missed
- Limited code coverage

Difference

Feature	White Box	Black Box
code Knowledge	Required	Not required
focus	Internal Logic	External behaviour
performed by	Developers	Testers/ users
Testing level	unit	system/ Acceptance

23/12/25 Dichotomies

The various dichotomies are as follows

① Testing Vs debugging
② Function Vs structure
③ The designer Vs the tester
④ Modularity Vs Efficiency
⑤ Small Vs large
⑥ The builder Vs the Buyer

• Dichotomies means it is the separation between two things they are similar and different Ex:- Like Tea and coffee.

① Testing Vs Debugging

Testing	Debugging
① The goal of testing is to detect errors in a program.	The goal of debugging is to detect error and correct them.
② Testing is initiated with Unknown condition	② Debugging is initiated with Known conditions
③ It is necessary to have planned, designed and scheduled constraints	③ while debugging it is not necessary to have these contraints
④ Testing is confirmation or perceptable occurancy	④ Debugging is a reduction process
⑤ Testing findout the reason for programmers failure	⑤ Debugging is a programmer's justification.
⑥ It is not necessary to have design knowledge while performing testing	⑥ It is sufficient to have detailed design knowledge for debugging
⑦ Testing is performed by a person who does not belong to the company.	⑦ Debugging should be done by a person who belong to the company and knowledge about outside environment
⑧ The test design and execution are done automatically	⑧ Design and execution cannot be done automatically -Ex.

② Function Vs Structure

Functional	Structure
① Functional Testing is treated as Black Box Testing.	① The structural testing is treated as white box testing
② It is the external behaviour of the system	② It is the internal behaviour of the system.
③ Both are useful, it have some limitations	③ Both are useful, it have some condition

④ They target different bugs, detect all bugs, but takes infinite time (More time)

④ Bugs cannot be completely detected

The designer Vs the tester

Designer	Tester
① The designer is based on the structural specification of the system	① Tester is based on the functional specification of the system.
② He/she depends on the implementation details	② He/she independent of the implementation details.
③ The knowledge of implementation makes the designer to design software i.e., efficient.	③ Due to the lack of implementation details the development software is inefficient
④ Software designers is responsible for design and executing the task	④ A tester is responsible for design and executing the task.

Small Vs Large

Small	Large
→ The small program have only few lines of a code	→ The large programs have large number of lines in a code.
② In small they consisting of few components.	② In large they consisting of large components.
③ The small programs does not required any technique for testing	③ They requires different types of techniques for testing.
④ It has following phases like desiging, coding and testing	④ It has a data dictionary to store the data. ~~and~~
⑤ In small single programmer.	⑤ In large different programmers.
⑥ High quality Ex:- Biggest of two numbers	⑥ Low quality. Ex:- Linked list
~~④ They can~~	

Modularity Vs Efficiency

Modularity	Efficiency
① Module is nothing but smallest component in a system.	① In modularity. integration process efficiency is required
② Smallest module integration is different but programming is understanding	② Smallest modules for less efficiency required.

③ Target Modules, integration is easy but programmer is not understanding

④ Each module has ~~huge~~ use independent test cases

③ Programmer's understanding very easy.

④ Large module for more efficient required.

The Builder Vs the buyer

Builder	Buyer
① who design the system and it's accountability	① who buys the system for profits, services

② The different roles/users in a system include

1. Builder : who designs the system and is accountable to the buyer.
2. Buyer :- who pays for the system in the hope of profits from providing services
3. user :- ultimate beneficiary or victim of the system. The user's interests are also guarded by.
4. Tester :- who is dedicated to the builder's destruction.
5. operator :- who has to live with the build

mistakes, the buyer's murky (unclear) specifications, tester's oversights and the user's complaints.

Modularity Versus Efficiency

A module is a discrete, well-defined, small component of a system. Smaller the modules, difficult to integrate; larger the modules, difficult to understand. Both tests and systems can be modular. Testing can and should likewise be organised into modular components. Small, independent test cases can be designed to test independent modules.

Date 24/12/25

The Consequence of Bugs

The consequence of bugs can range from mild to catagraphic, the programmers write programs for human and enhance the bugs consequence should be calculated in terms of human only rather than internal machine.

The consequence of bugs are as follows

① mild
② Moderate
③ Annoying
④ Distrubing

⑤ Serious
⑥ Very serious
⑦ Extreme
⑧ Intoderable
⑨ Catagraphic

Mild:- This consequence results in an incorrect misplaced or wrong aligned output.

Ex:- Results

```
1
1 2
1 2 3
1 2 3 4
```

change

```
1 2 3 4
1 2 3
1 2
1
```

wrong o/p / Error

Moderate:- This consequence effects the performance of the system and ends it's result in duplicate output.

Ex:- 1 2 3 (original) 0 1 2 3 (duplicate.)

Annoying:- This consequence effects in the system performance of system degrades.

① The names are shorted or changed.

② Builds for even a null amount are send

Ex:- SuperMarket → 1000 products, purchased 1 item, build generated 1 item

Multiple items like 1 2 3 --- 1000
only purchased 0 0 100 0
item remaining 0 → Bill generated

Disturbing:- This consequence effects even the correct transactions are not executed

Ex:- An automatic teller machine refuse to the process withdrawl transaction.

ATM having Money ← customer Sufficient withdraw Money

But correct transactions not executed.

Serious:- The information about the transaction get lost such as ① tracking of transactions
② Accountability (responsibility of a transaction
③ Transaction occurance
when such information is lost the resulting bug is called serious bug.

Ex:- A → B
A: 500
A sending amount to B
500 Already deducted
But not deposited in B account

Very serious:- This consequence results in reversing the operations of a transaction such as deposit transaction is converted into withdraw transaction 500 — where deducted we don't know.
enhance the system is forced to perform incorrect transactions.

A → Deposit amount in A's account
But deposit transaction show withdraw
Initially ₹ 5,00,000 → deposit 1,00,000
total — 6,00,000 serious bug
only 1 system deposit changed to withdraw 5−1 = 4
& withdraw can be deposit

Extreme:- This consequence occurs frequently and it is not limited (number of users)
↳ same problem for no. of systems.

Intoderable:- when a large data is corrupted it is very difficult to perform recovery process. In such situation shut down the system is considered as the best option.

SBI — Billions of users — error occur — all accounts deleted — shut down system.

Catagraphic:- This consequence the system is fail and hence the right to shut down is taken from the user.

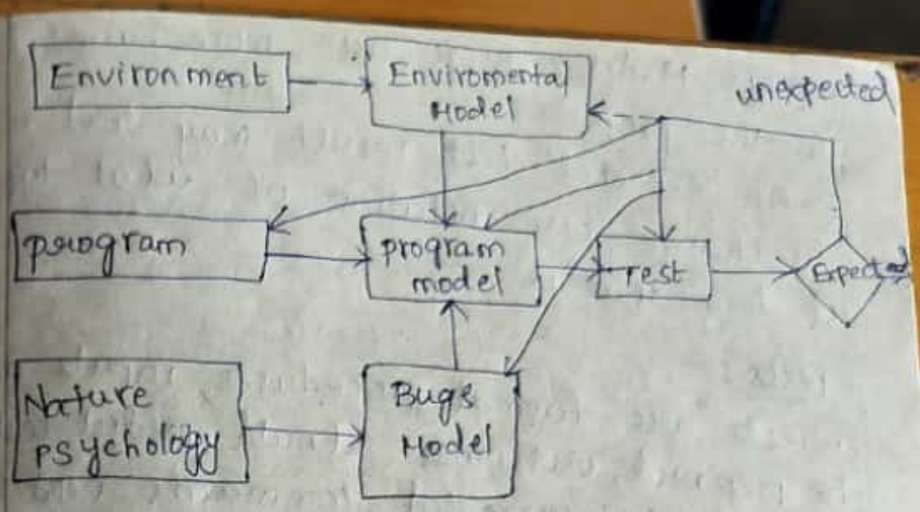
Date 29/12/25

Model For Testing

Testing is applied to any project sometimes it is very simple and sometimes it is very difficult.

The model for testing dividing into 3 modules

1. Environment module (Model)
2. Program model



3. Bug Model

Environment Model:- A program's environment is the hardware and software required to make it run.
for online system the environment may include communication lines, other systems, terminals and operators.
• The environment also include all programs that interact with and are used to create the program on that test such as operating system.

Program Module:- Most programs are too complicated to understand in detail.
→ The concept of the program is to be simplified in order to test it.
→ If a simple model of the program doesn't explain the unexpected mirror.

Bugs Model:- Bugs are more harmful to the project
→ An expected test result may lead us to change our notation of what type of bug that is.

Tests:-
- Tests are formal procedures, inputs must be prepared, outcomes should predict tests should be documented, commands need to be executed, and results are to be observed. All these errors are subjected to errors.
- We do three distinct kinds of testing on a typical software system. They are
1. unit / component testing:- A unit is the smallest piece of software that can be compiled, assembled, linked, loaded, etc
→ unit testing is the testing we do to show that that unit does not satisfy its function specification or that its implementation structure does not match the intended design structure
→ A unit testing usually work of one programmer and consists of several hundred or fewer lines of code.
→ A component is an integrated aggregate of one or more units.
→ Component testing is the testing we do to show that component does not satisfy its functional specification or that its implementation, structure does not match the intended design structure.
2. Integration Testing:- Integration is the process by which components are aggregated to create larger components. Integration testing is testing done to show that even though the components were individually satisfactory (after passing component testing), checks the combination of components are incorrect or inconsistent.
3. System Testing:- A system is a big component. System testing is aimed at revealing bugs that cannot be attributed to components. It includes testing for performance, security, accountability, configuration sensitivity, startup and recovery.

Roles of Models:- The art of testing consists of creating, selecting, exploring and revising models. Our ability to go through this process depends on the number of different models we have at hand and their ability to express a program's behaviour.

Date 30/12/25

Taxonomy of Bugs

The bugs are dividing into groups according to their behaviour, Problems and history.

→ Taxonomy of bugs are classified into 6 categories.

1. Requirements features and functional bugs.
2. Structural bugs
3. Data bugs
4. Coding bugs
5. Interface Integration and System bugs
6. Test and Test design bugs

Requirements features and functional bugs

- → Requirements and ~~classification~~ specification bugs
- → Feature bugs
- → Feature bugs interaction bugs

2) Structural Bugs

- → control and sequence bugs
- → Logic bugs
- → Processing bugs
- → Initialization bugs.
- → Data flow bugs

3) Data bugs

- → General
- → Dynamic vs static bugs
- → Information parameter and control
- → content structure Attributes.

4) Coding Bugs

5) Interface integration System bugs

- → External Interface
- → Internal interface
- → Hardware ~~area~~ architecture
- → OS bugs
- → Software Architecture
- → control of sequence
- → Resource Management

6) Test and Test design bugs

- → Testing
- → Test criteria

Requirements and Specification Bugs:-

→ Requirements and specifications developed from them can be incomplete ambiguous or self-contradictory. They can be misunderstood or impossible to understand.

→ The specifications that don't have flaws in them may change while the design is in Progress.

Feature Bugs:-
Specification problems usually create corresponding feature problems.
- A feature can be wrong, missing or superfluous (serving no useful purpose)
- Removing the feature might complicate the software, consume more resources and foster more bugs.

Feature Interaction Bugs:-
Providing correct, clear, implementable and testable features specifications is not enough.
- The problem is unpredictable interactions between feature groups or even between individual features.

Specification and feature Bug Remedies:-
Most feature bugs are rooted in human to human communication problems. One solution is to use high-level, formal specification languages or systems.

Control and Sequence Bugs:-
→ These bugs include paths left out, unreachable code, improper nesting of loops, loop-back or loop termination criteria incorrect, missing process steps, duplicated processing, unnecessary processing, rampaging GOTO's.

Logic Bugs:-
Bugs in logic, especially those related to misunderstanding how case statements and logic operators behave singly and combinations.

Processing Bugs:-
They include arithmetic bugs, algebraic, mathematical function evaluation, algorithm selection and general processing
Ex:- Improper usage of greater-than-or-equal.

Initialization Bugs:-
Initialization bugs are common. Initialization bugs can be improper and superfluous. Superfluous bugs are generally less harmful but can affect performance.

Data-Flow Bugs and Anomalies:-
Most initialization bugs are special case of data flow anomalies.
A data flow anomaly occurs where there is a path along which we expect to do something unreasonable with

data, such as using an uninitialized variable, attempting to use a variable before it exists, modifying and then not storing or using the result, or initializing twice without an intermediate use.

Data bugs:-
Data bugs include all bugs that arise from the specification of data objects, their formats.
→ They are at least as common as bugs in code, but they are often treated as if they did not exist at all.

Dynamic data vs static data:-
Dynamic data are transistory. whatever their purpose their lifetime is relatively short, typically the processing time of one transaction.
→ These are due to leftover garbage in a shared resource.
Static data are fixed in form and content. They appear in the source code or database directly or indirectly.

Information, Parameter and control:-
Static or dynamic data can serve in one of three roles, or in combination of roles as a parameter, for control, or for information.
→ content can be an actual bit pattern, character string or number put into data structure.
→ structure relates to the size, shape and numbers that describe the data object which is memory location used to store the content.
→ Attributes relates to the specification meaning that is the semantics associated with the contents of data object.

coding bugs:-
coding errors of all kinds can create any of the other kind of bugs.

Interface, integration and system bugs:-
There are various categories in it.
External interfaces:-
They are the means used to communicate with the world. They include devices, actuators, sensors, input terminals, printers and communication lines.

Internal interfaces:-
↳ They are in principle not different from external interfaces but they are more controlled.
→ The best example is communicating routines.

Hardware Architecture :-
Bugs related to hardware architecture originate mostly from misunderstanding how the hardware works.
Examples include address generation error, i/o device operation, instruction error, waiting too long for a response, incorrect interrupt handling etc.

Operating System Bugs:-
program bugs related to the operating system are a combination of hardware architecture and interface bugs mostly caused by a misunderstanding of what is the operating system does.

Software Architecture:-
↳ The bugs are the kind of that called interactive.
→ Many of them depend on load, and their Symptoms emerge only when the system is stressed.

Control and Sequence Bugs:-
These bugs include : ignored timing, assuming that events occur in a specified sequence, working on data before all the data arrived from disk, waiting for impossible prerequisites, missing, wrong.

Resource Management Problems :
Memory is subdivided into dynamically allocated resources such as buffer blocks, queue blocks, task control blocks and overlay buffers.
Resource management remedies:- A design remedy that prevents bugs is always preferable to a test method that discovers them. It is to keep the structure simple, with few kinds of resources.

Integration Bugs:-
They are bugs having to do with the integration of, and with the interfaces between, working and tested components. These bugs results from inconsistencies or incompatibilities between components.

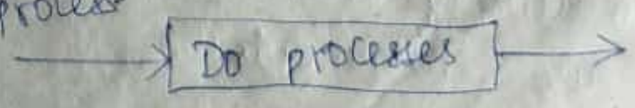
System Bugs:-
System bugs covering all kinds of bugs that cannot be ascribed to a compon or to their simple transactions, but result from the totality of interactio between many components such as program, data, hardware and the operating system.

Test and Test Design Bugs:-
Testing:- Testers have no immunity to bugs. Tests require complicated scenario and databases.
They require code or the execute and consequently they can have bugs.
Test criteria: If the specification is correct, it is correctly interpreted and implemented, a proper test case is designed.

31/12/24

Basic concepts of path testing
① 'path testing used' in white box testing and unit testing
* The path testing is introduced by Tom

Control Flow Graph
A control flow graph is a form of flow chart which does not deals with internal structure of the process rather than it show the data flow and control flow between the process
→ Every control flow graph has some elements:
① The process Blocks
② Decision and case statements
③ Junctions
The process Blocks :- process block is a block which consisting of series of statements once initiated the results are execute all the statements in a process block.
→ A process block are neither invented by decisions or junctions.
→ Every process block has an entry, an exist consisting of a single or series of statements.



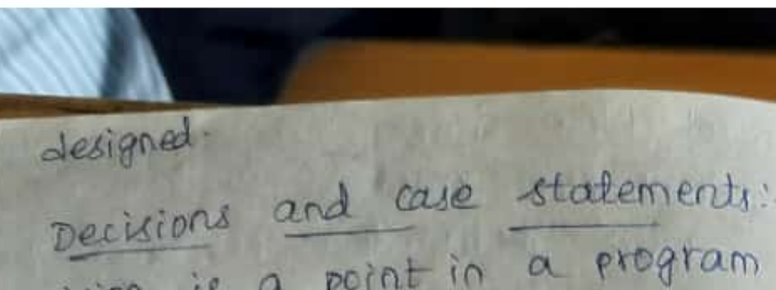
Control flow graph are not connected with the details of operations in a process block so that test cases are

designed.

Decisions and case statements:-

Decision is a point in a program at which the control flow can split.

* The flow gets diverted into one or many options available.
* Mostly decision split into two way & binary occasionally they split three way branches.
* The test case designs are easy for two way and three way branches.

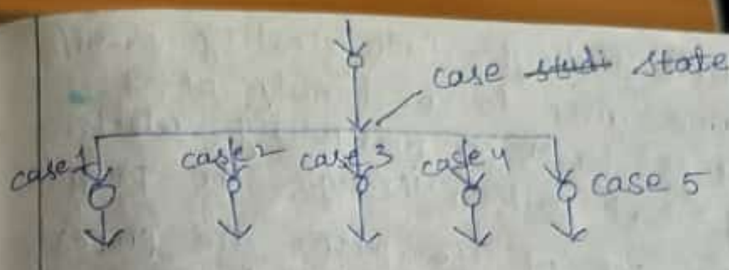


Two way Decision (Binary)

Case statements:- Any decision can split the control flow into different waye branches.

This multiway branches can be termed as case statements

The representation of case statement is shown →

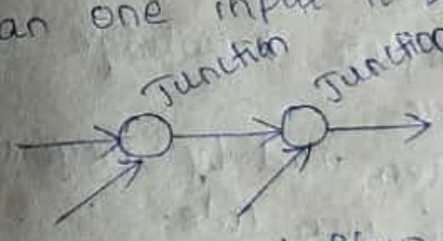


The designing of test cases for decision and case statements are same

Junction:- A junction is just the reverse of the decision

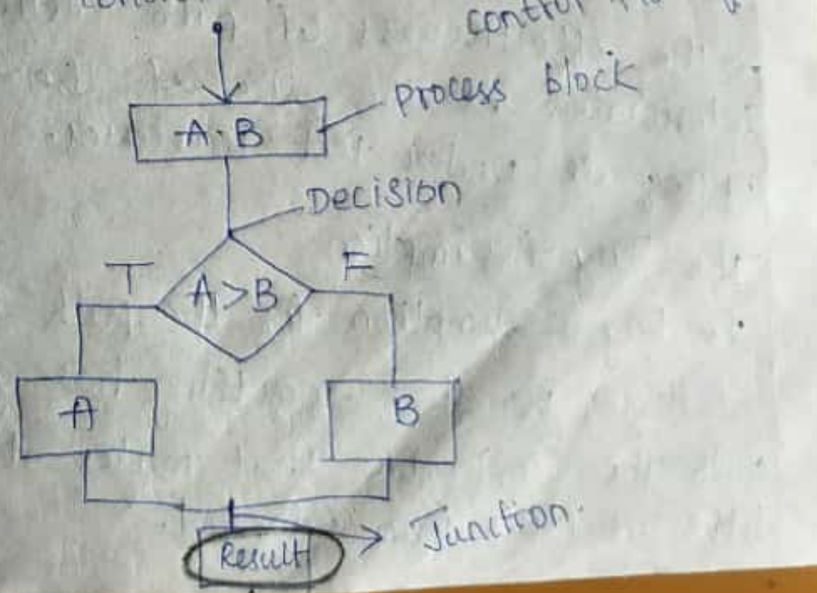
→ All the control flow can merge at a one point in program which can be termed as junction

→ In other words a node with more than one input it is called as junction.



Ex: control flow

Find out biggest of 2 no's by using control flow graph



Date 2/11/26

Path Testing:- Path testing is the name given to a family of test techniques based on judiciously selecting a set of test paths through the program.

→ If the set of the paths are properly chosen then we have achieved some measure of test thoroughness. for example pick enough paths to assure that every source statement has been executed at least once.

→ Path testing techniques are the oldest of all structural test techniques.

→ Path testing is most applicable to new software for unit testing. It is a structural technique.

→ It requires complete knowledge of the program's structure.

→ It is most often used by programmers to unit test their own code.

→ The effectiveness of path testing rapidly deteriorates as the size of the software aggregate under test increases.

The Bug Assumption

The bug assumption for the path testing strategies is: that something has gone wrong with the software that makes it take a different path than intended.

As an example "GOTO x" where "GOTO y" had been intended.

→ structured programming languages prevent many of the bugs targeted by path testing; as a consequence the effectiveness for path testing for these languages is reduced and old code in COBOL, ALP, FORTRAN and BASIC, the path testing is indispensible.

Control flow graphs Vs flowcharts

→ A program's flow chart resembles a control flow graph.

→ In flow graphs, we don't show the details of what is in a process block.

→ In flow charts every part of the process block is drawn.

→ The flowchart focuses on process steps, where as the flow graph focuses on control flow of the program.

→ The act of drawing a control flow graph is a useful tool that help us clarify the control flow and data flow issues.

Notational Evolution:-

The control flow graph is simplified representation of the program's structure. The notation

changes made in creation of control flow g
. The process boxes weren't really needed
is an implied process on every line joinin
junctions and decisions.
. We don't know the specifics of the decisio
need to
just the fact that there is a branch.
→ The specific target label names aren't
important - just the fact that they exist; &
we can replace them by simple numbers.
To understand this, we will go through an
example (Figure 2.2) written in a FORTRAN
like programming language called programming
Design Language (PDL). The program's
corresponding flowchart (Figure 2.3) and
flowgraph (Figure 2.4) were also provided
below for better understanding.
The first step in translating the program to
a flowchart is shown in figure 2.3, where
we have the typical one-for-one classical
flowchart. Note that complexity has increased,
clarity has decreased, and that we had
to add auxiliary labels (LOOP, XX and YY),
which have no actual program counterpart.
In figure 2.4 we merged the process steps
and replaced them with the single process
box.

We now have a control flow graph. But
this representation is still too busy. We
simplify the notation further to achieve
Figure 2.5, where for the first time we
can see really what the control flow
looks like.

```
                CODE * (PDL)
INPUT X, Y                      V(U-1):=V(U+1)+V(V-1)
Z:= X+Y                         ELL: V(U+V(V)):=U+V
V:= X-Y                              IF U=V GOTO JOE
IF Z>=0 GOTO SAM                     IF U>V THEN U:=Z
JOE: Z:= Z-1                              Z:=V
SAM: Z:= Z+V                              END
  FOR U=0 TO Z
V(U), U(V):=(Z+V)*U
  IF V(U)=0 GOTO JOE
  Z:=Z-1
IF Z=0 GOTO ELL
  U:=U+1
  NEXT U
```

· A contrived horror
Figure 2.2 Program Example (PDL)

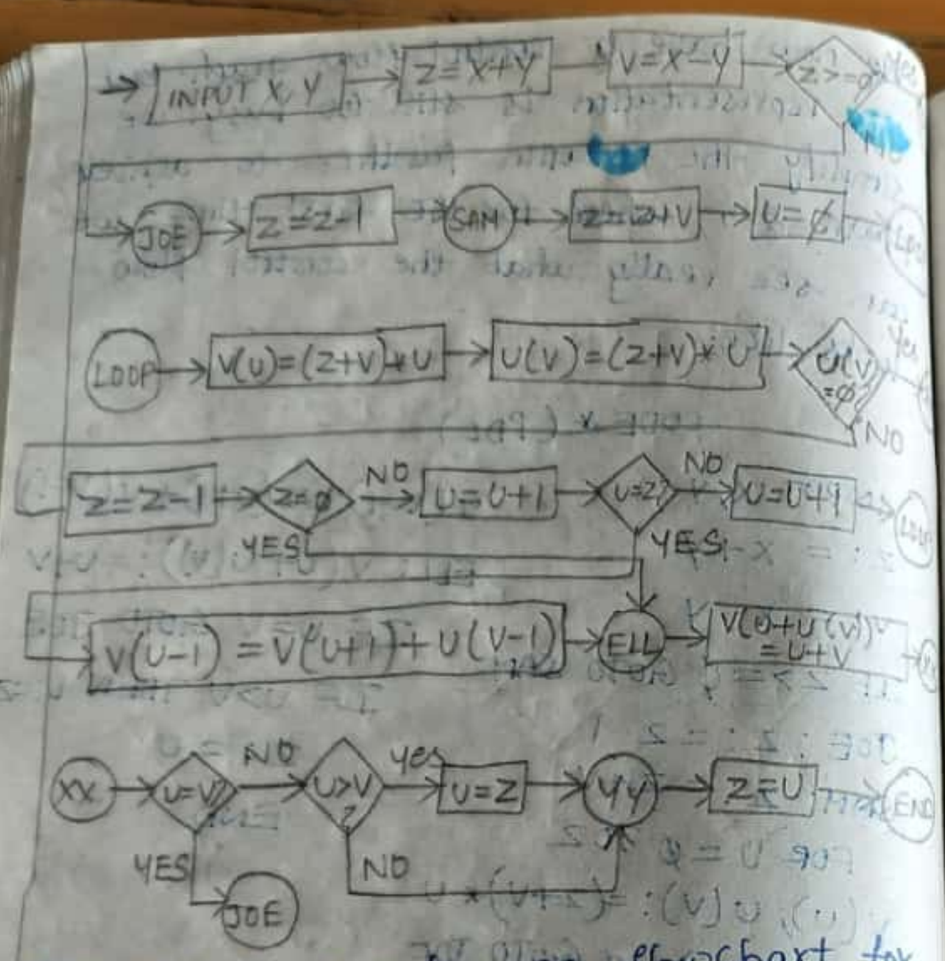


Figure 2·3: One-to-one flowchart for example program in Figure 2·2

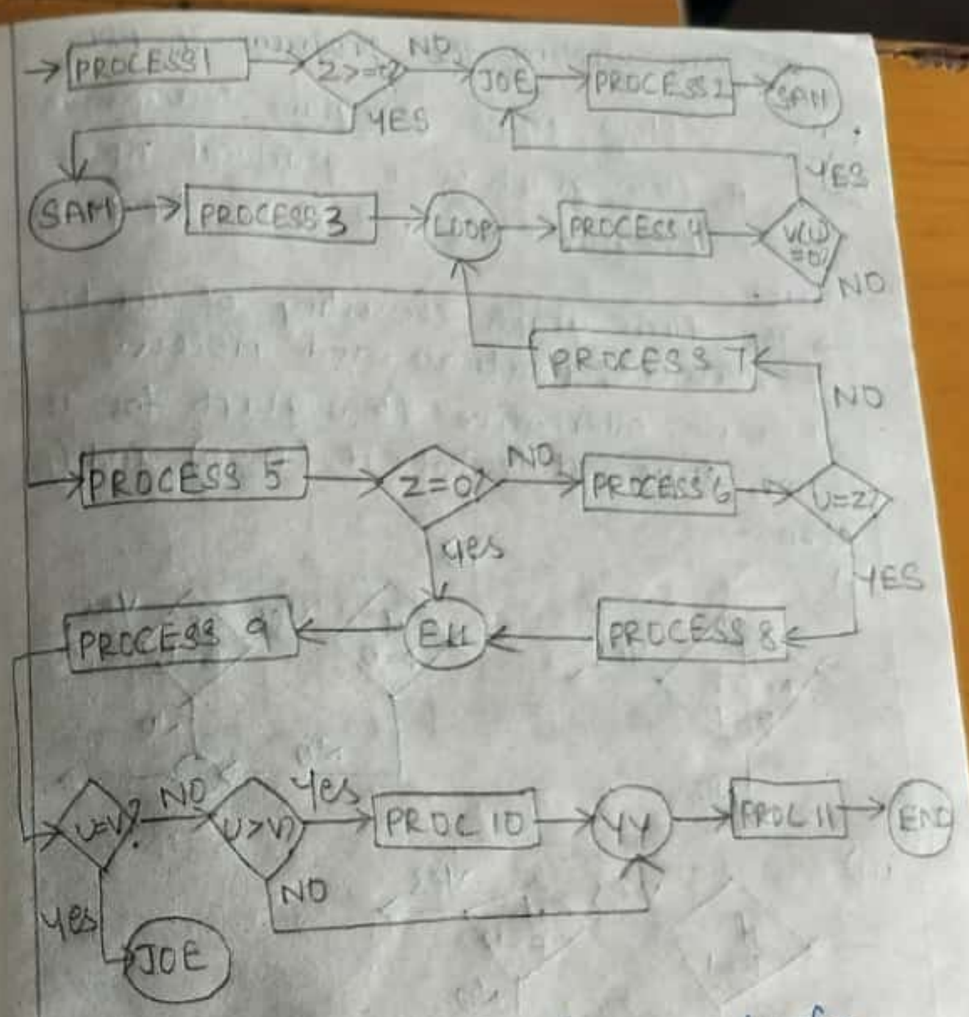


Figure 2·4: Control Flow graph for example in Figure 2·2

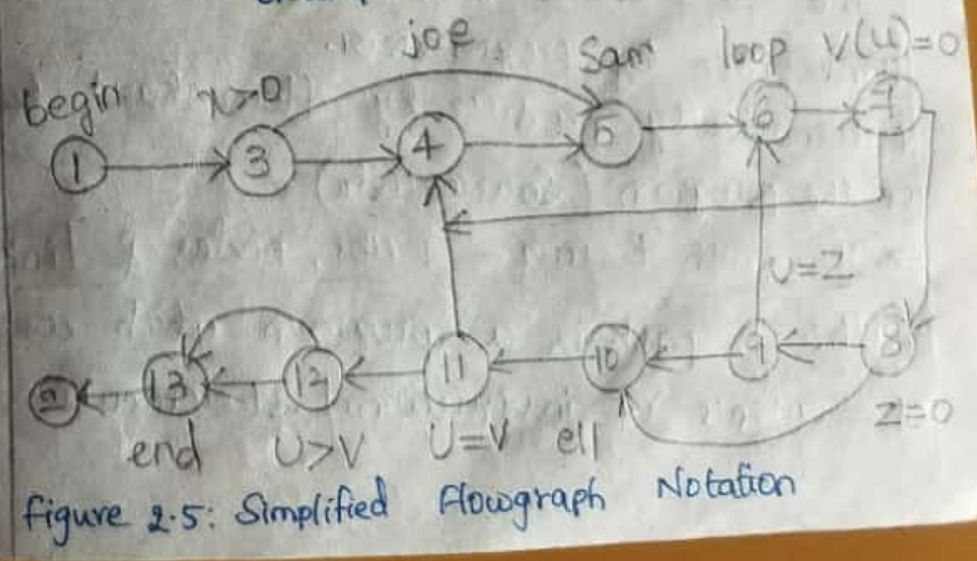


Figure 2·5: Simplified Flowgraph Notation

we are writing the program in PDL.

Date 5/1/26

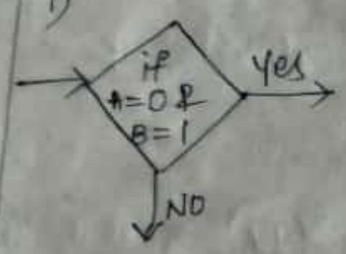
Flow graph program correspondance

A flow graph is a graphical represent of a program and just it is topographi map.

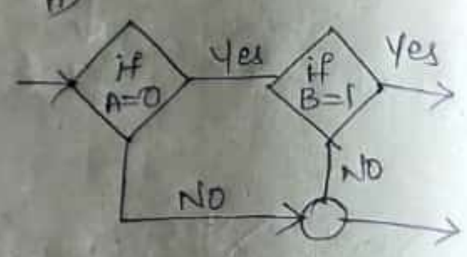
→ The flow graph consisting of combinat of decisions, junctions and process.

→ Example: Alternative flow graph for the same logic, statement if (A=0) and (B=1 then ....

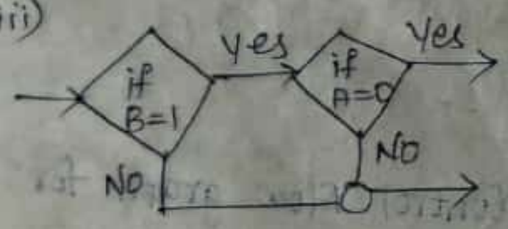
i)



ii)



iii)



The flow chart & flow graph generation Handwritten by programmer.

Automatically produce by flow chart based on the program (Source code)

The path testing - Paths, nodes & links

path:- A path is a program which consisting of sequence of instructions/ statements.

→ Every path consisting of starting point and an exit point and it process decision junctions and case statements.

→ A path may go through several junction process, junctions one or more times.

→ path consisting of segments. Segment is nothing but it's a link connecting in between the two nodes.

→ The length of the path measured by the number of links in it and not by the number of instructions or the statements execute along with the path.

The fundamental path selection criteria

→ Exertise every path from entry to exit

→ Exercise every statement or instructions atleast once.

→ Exercise every branch and case statements in each direction atleast once.

Case 1:- if (X, LT, 0) go to 200

X = X + A

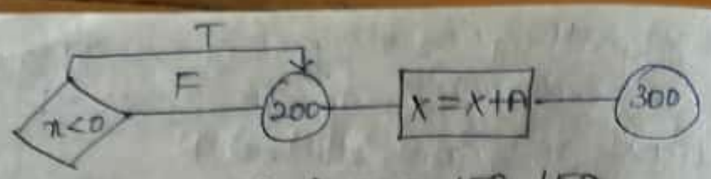
200 X = X + A

Case 1:- 300 continue

Case 2:- if (X, LT, 0) go to 200

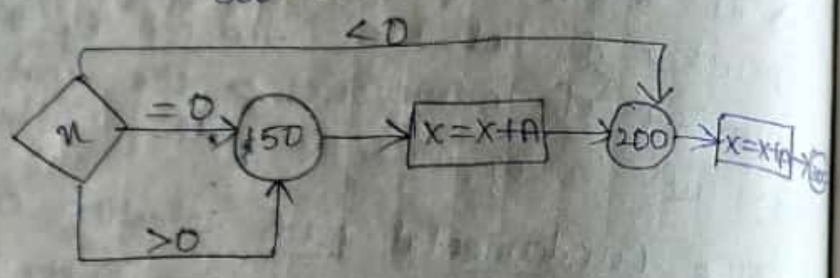
200 X = X + A

300 continue



Case 3:- If (x) 200, 150, 150

```
100   x = x+A
      goto 100
150   x = x+A
200   x = x+A
300   continue
```



Date 6/1/26

path testing criteria

we have three types of testings

i) Branch testing ii) statement testing iii) Path testing

path testing :- Execute all possible control flow graph through the program.

→ All possible entry & exit path through the program.

→ If you achieve this perception we are said to achieve hundred percent path coverage

This is strong criteria in the path testing strategies in family.

It is generally impossible to achieve.

statement testing (C1):- Execute all the statements in the program atleast once under some test.

→ If you do enough test to achieve this we are said to achieve 100% statement coverage.

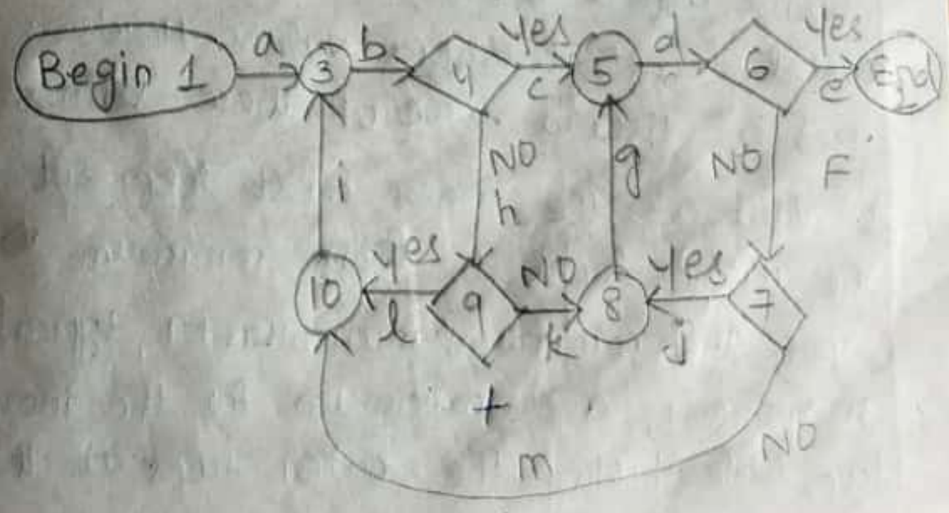
→ If is the weakest strategy in the family

→ are denote this by C1

Branch testing (C2):- Execute enough tests to ensure that every branch alternative has been exercised atleast once under some test. If you do enough test to achieve this perception then we achieved 100% branch coverage (link coverage)

→ we denote branch coverage by C2

path selection example



Path	Decision				process link									
	4	6	7	9	a	b	c	d	e	f	g	h	i	j
abcde	Y	Y	N	N	✓	✓	✓	✓	✓					
abhkgde	N	Y	N	N	✓	✓		✓	✓		✓	✓		
abhlibcde	Y	Y	N	Y		✓	✓	✓	✓			✓	✓	
abcdfjgde	Y	Y	Y	N	✓	✓	✓	✓	✓	✓			✓	
abcdfmibcde	Y	Y	N	N	✓	✓	✓	✓	✓	✓			✓	

Kinds of Loops:- There are only three kinds of loops with respect to path testing

- Nested Loops-

The number of tests to be performed on nested loops will be the exponent of the tests performed on single loops. As we cannot always afford to test all combinations of nested loops' iterations values. Here's a tactic used to discard some of these values

1. Start at the inner most loop. Set all the outer loops to their minimum values
2. Test the minimum, minimum+1, typical, maximum-1, and maximum for the innermost loop, while holding the outer loops at their minimum iteration parameter values. Expand the tests as required for out of range and excluded values.
3. If you've done the outmost loop, GOTO step 5, else move out one loop and set it up as in step 2 with all other loops set to typical values.
4. Continue outward in this manner until all loops have been covered.
5. Do all the cases for all the loops in the nest simultaneously.

Concatenated Loops:-

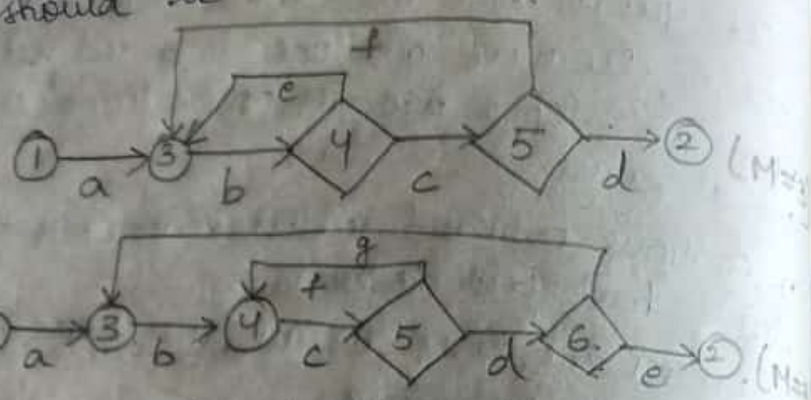
Concatenated loops fall between single and nested loops with respect to test cases. Two loops are concatenated if it's possible to reach one after existing the other while still on a path from entrance to exit.

If loops cannot be on the same path, then they are not concatenated and can be treated as individual loops.

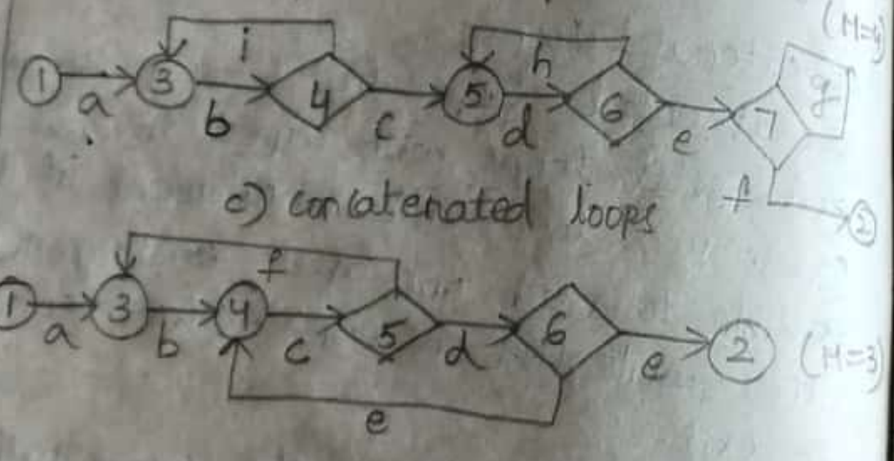
Horrible Loops:-

A horrible loop is a combination of nested loops, the use of code that jumps into and out of loops, intersecting loops, hidden loops, and cross connected loops.

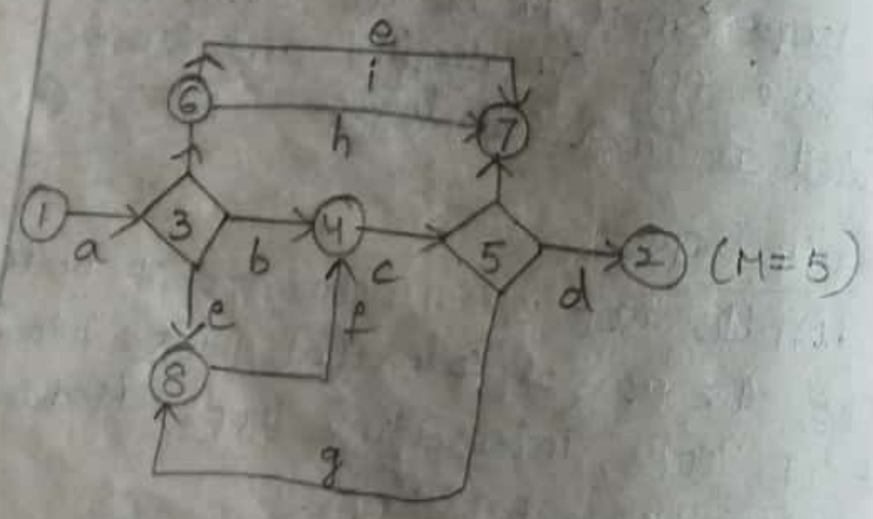
Makes iteration values selection for te
cases an awesome and ugly task, whi
is another reason such structures
should be avoided.



a,b) Nested loops



c) Concatenated loops



d,e,f) Horrible loops

Figure 2.10: Example of loop types

possible paths

		Decisions
· abcde	· abckmie	
· abcghe	· abcgefcde	1, 5, 7, 8
· almie	· abckmjcde	

Predicate, Path Predicate & achievable paths

Predicate:- Decision is a point in a program which the control flow gets splitted.

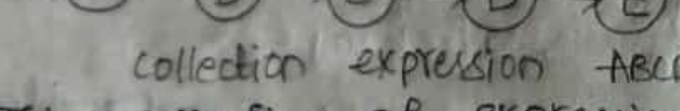
- Predicate is a function which is logically executing during the decision processing
- The result of the function decides the direction of the flow (process block execute)
- If the flow is positive every process block, every software, every application system software is executed.

• If the flow is negative, it is not execute any program.

Ex:- A ≤ 5
```
if (A ≤ 5)
    printf(" ")
else
    print(" ")
```

predicate expression

• Path predicate expressions are the collection of expressions that must be fulfilled in order to achieve the desired path.



collection expression ABCDE of path

• This collection of expressions are satisfied based on the input values provided, this input values must met all the expressions.

• If all the expressions are met then the path is chosen else the path is rejected

Date 8/1/26

Predicate coverage :- It is the process of testing all the truth values related to a specific path in all possible ways

→ All possible ways means process blocks, decisions, functions must executed.

→ If all the values are tested in all possible directions then we can say that 100% predicate coverage is achieved.

Testing Blindness :-

Blindness is a situation which results in the correct path via wrong route on it.

There are 3 types of predicate blindness they are as follows :-

i) Assignment Blindness
ii) Equality blindness iii) Self blindness

Assignment blindness :- It comes into consideration when both predicates irrespective of their correctness are satisfied by a value assigned to the assignment statement. It leads to wrong path selection.

Example :- In the program we are assigned a=5 then we can use a value as 5 in entire the program but sometime it is taken 'a' value is 6.

Equality blindness :- whenever a path is chosen by the predicate some values are generated as an outcome, if that value satisfied the consecutive predicate irrespective of it's correctness then it is called equality blindness.

Ex:- In the program (sum of 2 no's)

here we can assign a=5 b=10 the outcome is a+b=15 but it will n come

Self Blindness:- sometimes both correct and wrong predicates became similar to each other depending on the values assigned to the assignment statement

Ex:- wrong predicate 5 correct predicate 0005

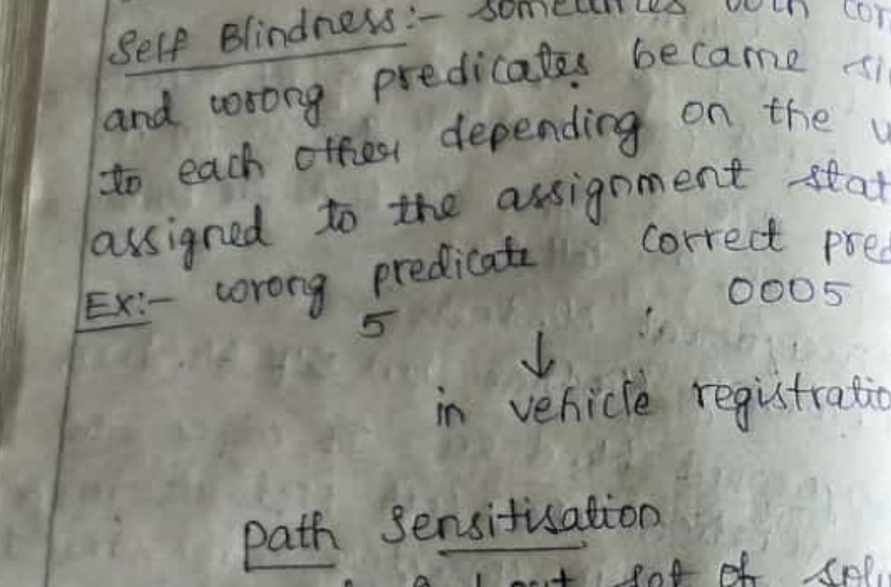
↓

in vehicle registration.

Path Sensitisation

An attempt to find out set of solution for path predicate expression that is called the path sensitisation.

The best procedure the Heuristic procedure



Path	predicate values
nopqrs	$P\bar{R}$
ntupqrs	$\bar{P}QR$
ntyzwrs	$\bar{Q}S$
nopvzwrs	$PR\bar{S}$
nopvzrs	PR
ntupvzrs	$\bar{P}QRS$
ntupvzwrs	$\bar{P}QR\bar{S}$

9/1/26

Heuristic procedures for sensitizing paths

1. This is a workable approach, instead of selecting the paths without considering how to sensitize, attempt to choose a covering path set that is easy to sensitize and pick hard to sensitize paths only as you must to achieve coverage
2. Identify all variables ~~as dependent or in~~ that affect the decision
3. classify the predicates as dependent or independent
4. Start the path selection with un corelated independent predicates
5. If coverage has not been achieved using independent uncorelated predicates, extend the path set using co-related predicates.
6. If coverage has not been achieved extend the cases to those that involve dependent predicates.
7. Last, use corelated, dependent predicates.

Path Instrumentation:-
path instrumentation is what we have to do to confirm that the outcome was achieved by the intended path.

Co-incidental correctness:- The coincidental correctness stands for achieving the desired outcome for wrong reason.

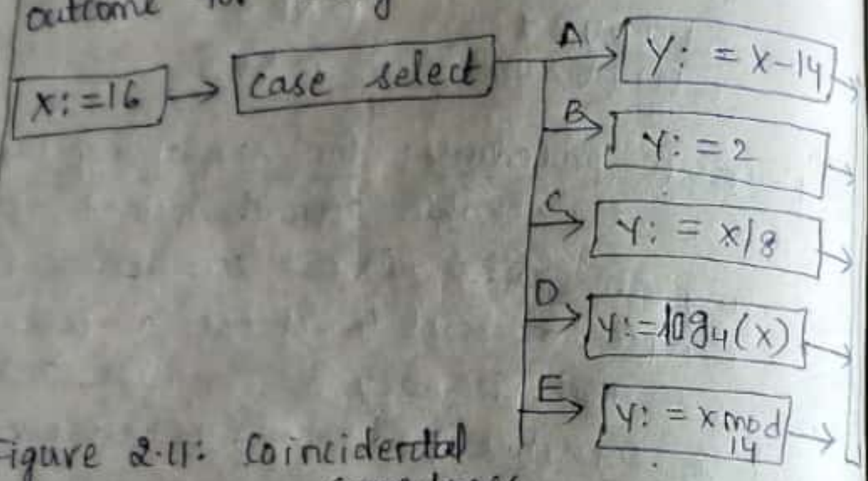


Figure 2.11: Coincidental correctness.

The above figure is an example of a routine that, for the (unfortunately) chosen input value (X 16), yields the same outcome (Y=2) no matter which case we select. Therefore, the tests chosen this way will not tell us whether we have achieved coverage. For example, the five cases could be totally jumbled and still the outcome would be the same. Path Instrumentation is what we have to do to confirm that the outcome was achieved by the intended path.

The types of instrumentation methods include:

1. Interpretive Trace program:-
An interpretive trace program is one that executes every statement in order and records the intermediate values of all calculations, the statement labels traversed etc.
→ If we run the tested routine under a trace, then we have all the information we need to confirm the outcome and, furthermore, to confirm that it was achieved by the intended path.
→ The trouble with traces is that they give us far more information than we need. In fact, the typical trace program provides so much information that confirming the path from its massive o/p dump is more work than simulating the computer by hand to confirm the path.

2. Traversal Maker (or) Link Marker:-
A simple and effective form of instrumentation is called a traversal maker or link marker.
- Name every link by a lower case letter.
- Instrument the links so that the link's name is recorded when the link is executed

The succession of letters produced in q from the routine's entry to its exit should, if there are no bugs, exactly co to the pathname.

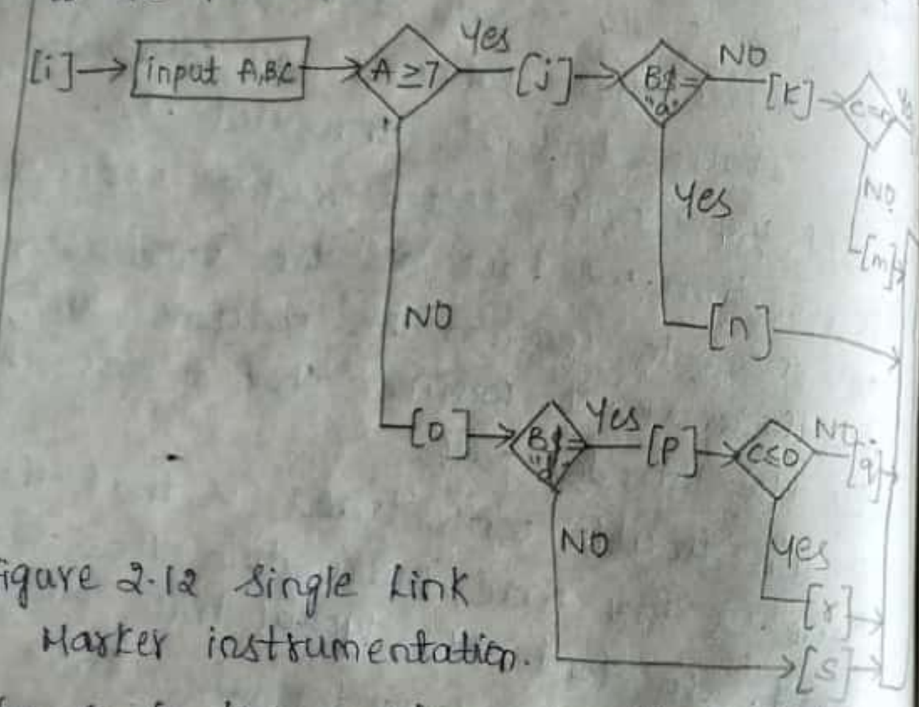


Figure 2.12 Single Link Marker instrumentation.

Why single link markers aren't enough.

Unfortunately, a single link marker may not do the trick because links can be chewed by open bugs.

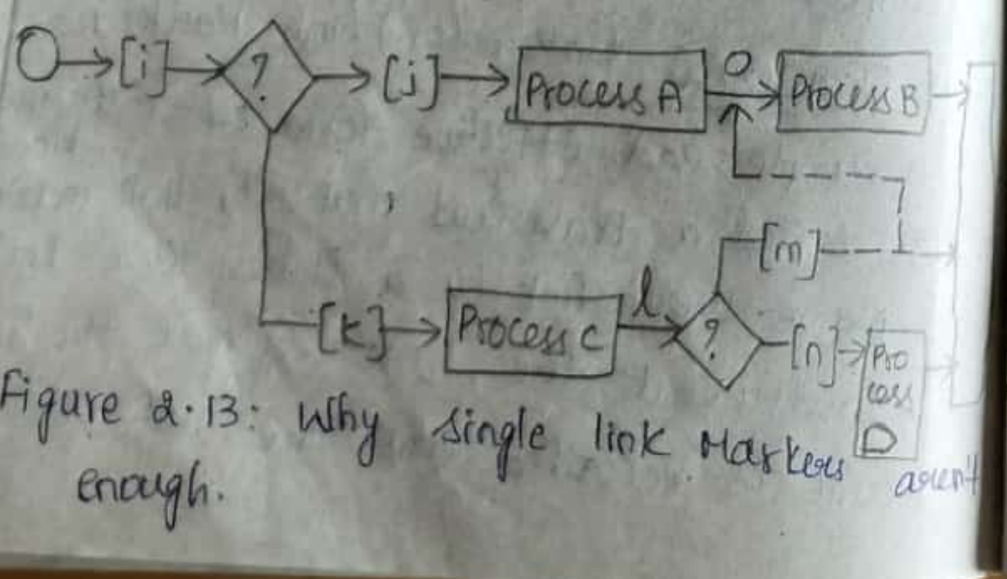


Figure 2.13: Why single link Markers aren't enough.

We intended to traverse the ikm path, but because of a rampaging GOTO in the middle of the m link, we go to process B. If coincidental correctness is against us, the outcomes will be the same and we won't know about the bug.

Two Link Marker Method:-

The solution to the problem of single link marker method is to implement two markers per link: one at the beginning of each link and on at the end.

The two link markers now specify the path name and confirm both the beginning and end of the link.

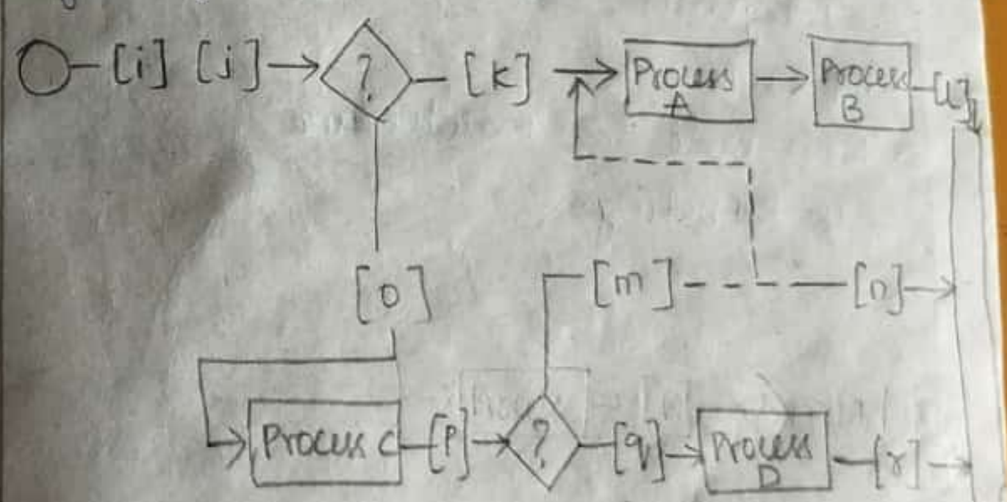


Figure 2.14: Double link marker instrumentation.

Link counter: A less disruptive (and less informative) instrumentation method is based on counters. Instead of a unique link

name to be pushed into a string when the link is traversed, we simply increment the link counter. We now confirm that the path length is as expected. The same problem that led us to double link markers also leads us to double link counters.

Applications of path testing:-

→ Path testing is a process which involves all the available paths in a program from an entry to an exit in such a way that the entire path is thoroughly tested.

→ Path testing implementation and applications can be categorized as follows.

1. Integration, coverage and paths in called components.
2. New node 3. Maintenance
4. Rehosting.

10/1/26

A transac
from syst
→ A transa
operations
by a sep
outside of
Transaction
created a
points.
→ At the
processing
the systen

Examples
A transact
retrival sy
following
1. Accept
2. Validate
3. Transmit
4. Do inpu
5. Search f
6 Request di
7 Accept inp
8 Validate in